

{EPITECH}

JOB BOARD

< BOOTSTRAP />



JOB BOARD

Building a job board means wiring together three moving parts that must agree with each other from day one: a relational database, a REST API, and a dynamic front-end.

Get any one of these wrong early — a schema that doesn't match what the UI needs, an API that ignores REST conventions, a front-end that reloads the page instead of fetching data — and you'll pay for it later, across every single step of the project.

In this bootstrap, you'll practice, on a small guided example, the **exact same tools** you'll use in the project: SQL modeling, a RESTful API, and asynchronous JavaScript (`fetch()`). You'll also break a few of these things on purpose, so you recognize the symptoms when they show up in your own project.

Data outlives code, and APIs outlive front-ends. Design the boring parts first.
— A rule of thumb worth stealing —

Database Modeling

A recruiting agency called HireLoop is about to launch a tiny job board. Before touching any code, they hand you a rough, messy spec:

- ✓ companies post job ads,
- ✓ job ads belong to exactly one category,
- ✓ people can apply to several ads, and one ad can receive several applications,
- ✓ each application should keep a trace of the message sent and the date.

Create the tables in SQL and insert a handful of fake rows by hand (2 companies, 3 ads, 3 people, a few applications) so you have something to query.

Write three SQL queries to answer questions such as:

- ✓ how many ads has each company posted?
- ✓ which ad has received the most applications?
- ✓ list all applicants for a given ad, with their message.



If a query needs a JOIN across three tables to answer a one-line question, that's normal here — it's exactly the kind of query your future API routes will run.

REST API

HireLoop now wants a small API in front of that database. You don't need to cover every table — a couple of resources is enough to practice the pattern properly.

Build at least these routes, on top of your schema:

- ✓ **GET /companies/:id/ads** — list all ads posted by a company;
- ✓ **GET /ads/:id** — return the full detail of one ad;
- ✓ **POST /ads** — create a new ad;
- ✓ **POST /ads/:id/applications** — record a new application for that ad.



Routes name resources, not actions. `/ads/:id/applications` is RESTful; `/createApplication?adId=...` is not.

Use the verb (GET/QUERY/POST/PUT/PATCH/DELETE) to say what you're doing, and the URL to say what you're doing it to.

For each route, make sure the status code actually matches what happened:

- ✓ 200 / 201 for a normal success (201 when something new was created),
- ✓ 400 if the request body is missing a required field,
- ✓ 404 if the id in the URL doesn't exist.

Full list of HTTP status codes: [MDN — HTTP response status codes](#)



Test every route with a REST client (curl, Postman, Insomnia, or your framework's test UI) before writing a single line of front-end code. If a route is wrong, it's much cheaper to fix it now than after the UI depends on it.

Dynamic Front-end

Build one minimal HTML/CSS page for HireLoop that:

- ✓ Fetches the list of ads from your API when the page loads;
- ✓ Displays each ad's title and a short description;
- ✓ Shows the full detail of an ad when a "View details" button is clicked — without reloading the page.

This means using `fetch()` (or an equivalent), handling the response asynchronously, and updating the DOM directly (`innerHTML`, `createElement...`) rather than navigating to a new page.



If clicking "View details" makes the URL bar change or the whole page flash/reload, you're not there yet — that's a symptom you'll want to recognize and fix quickly in the real project.

v 0.0.1

{EPITECH}